

Q21. Application: Autonomous Vehicle Navigation System

Aim:

To solve the recurrence relation

$$T(n)=2T(n/2)+n^2$$

using the Master Theorem, derive its time complexity, and analyse its scalability in an autonomous vehicle navigation system.

Algorithm:

1. Identify the recurrence relation.
2. Compare it with the standard Master Theorem.
3. Find the values of a, b, and $f(n)$.
4. Compute $n \log_b a$.
5. Select the appropriate Master Theorem case.
6. Derive the asymptotic time complexity.
7. Analyse scalability using asymptotic notation.

Source Code:

```
main.c    Share  Run
1  #include <stdio.h>
2  int T(int n)
3  {
4      if (n <= 1)
5          return 1;
6      return 2 * T(n / 2) + n * n;
7  }
8  int main()
9  {
10     int n;
11     printf("Enter the value of n: ");
12     scanf("%d", &n);
13     printf("Result T(%d) = %d\n", n, T(n));
14     return 0;
15 }
```

Step-by-Step Analysis:

Given,

$$T(n)=2T(n/2)+n^2$$

Compare with the standard Master Theorem:

$$T(n)=aT(n/b)+f(n)$$

Here,

- $a=2$
- $b=2$
- $f(n)=n^2$

Now,

$$\log_b a = \log_2 2 = 1$$

Therefore,

$$n \log_b a = n$$

Since,

$$f(n) = n^2 = \Omega(n^{1+\epsilon}), \epsilon = 1$$

Also,

$$2f(n/2) = 2(2n)^2 = 2n^2 \leq cf(n)$$

where $c = 1/2 < 1$.

Hence, the recurrence satisfies Master Theorem Case 3.

Therefore,

$$T(n) = \Theta(n^2)$$

Master Theorem Justification:

- $a = 2$
- $b = 2$
- $f(n) = n^2$
- $n \log_2 2 = n$
- Since $f(n)$ grows faster than n , Case 3 applies.

Hence,

$$T(n) = \Theta(n^2)$$

Time Complexity:

$$\Theta(n^2)$$

Space Complexity:

Recursive depth:

$$\log_2 n$$

Therefore,

$$O(\log n)$$

Scalability Analysis:

- The navigation system recursively divides the map into two equal regions.
- Each level performs n^2 route-processing operations.
- As map data increases, execution time grows quadratically.
- Suitable for medium-sized road networks but becomes computationally expensive for very large maps.

Asymptotic Interpretation:

- **Best Case:** $O(n^2)$
- **Average Case:** $\Theta(n^2)$
- **Worst Case:** $\Omega(n^2)$

Hence,

$$T(n) = \Theta(n^2)$$

Sample input and output:

```
Output
Enter the value of n: 8
Result T(8) = 120

=== Code Execution Successful ===
```

Q23. Application: Weather Forecasting Simulation System

Aim:

To analyse the algorithms $O(n \log n)$ and $O(n^2 \log n)$ using asymptotic notation, compare their growth rates, and determine the most efficient algorithm for large-scale weather forecasting simulations.

Algorithm:

1. Identify the asymptotic complexity of both algorithms.
2. Compare their growth rates.
3. Analyse their scalability.
4. Determine which algorithm performs better.
5. Interpret the result using asymptotic notation.

Source code:

```
main.c    Share  Run
```

```
1  #include <stdio.h>
2  #include <math.h>
3  int main()
4  {
5      int n;
6      printf("Enter the value of n: ");
7      scanf("%d", &n);
8      double algorithm1 = n * log2(n);           // O(n log n)
9      double algorithm2 = n * n * log2(n);       // O(n^2 log n)
10     printf("\nAlgorithm 1 (O(n log n)) = %.2lf\n", algorithm1);
11     printf("Algorithm 2 (O(n^2 log n)) = %.2lf\n", algorithm2);
12     if (algorithm1 < algorithm2)
13         printf("\nAlgorithm 1 is more efficient for large-scale weather\n\nforecasting.\n");
14     else
15         printf("\nAlgorithm 2 is more efficient.\n");
16     return 0;
17 }
```

Step-by-Step Analysis:

Algorithm 1:

$$T1(n)=O(n\log n)$$

- Running time increases almost linearly.
- The logarithmic factor grows very slowly.
- Efficient for processing large weather datasets.

Algorithm 2:

$$T2(n)=O(n^2\log n)$$

- Running time increases quadratically.
- The logarithmic factor further increases execution time.
- Suitable only for smaller datasets.

Time Complexity:

- Algorithm 1: $O(n\log n)$
- Algorithm 2: $O(n^2\log n)$

Scalability Analysis:

- Algorithm 1 scales efficiently as weather data increases.
- Algorithm 2 requires much more computation for large datasets.
- Large-scale forecasting systems benefit from lower growth rates.

Asymptotic Interpretation:

Algorithm 1:

- Best Case: $O(n\log n)$
- Average Case: $\Theta(n\log n)$

- Worst Case: $\Omega(n \log n)$

Algorithm 2:

- Best Case: $O(n^2 \log n)$
- Average Case: $\Theta(n^2 \log n)$
- Worst Case: $\Omega(n^2 \log n)$

Conclusion:

Since,

$$O(n \log n) < O(n^2 \log n)$$

the $O(n \log n)$ algorithm is the most suitable for large-scale weather forecasting simulations because it has a significantly lower growth rate, executes faster, and scales efficiently as the amount of weather data increases. This answer follows the same step-by-step style as the Master Theorem examples in your uploaded document.

Sample input and output:

```
Output
Enter the value of n: 16

Algorithm 1 (O(n log n))    = 64.00
Algorithm 2 (O(n^2 log n)) = 1024.00

Algorithm 1 is more efficient for large-scale weather forecasting.

=== Code Execution Successful ===
```